ILP-Based Scheduling for High-Level Synthesis

Md Rafiul Islam
ID: 1232714
Hamm-Lippstadt University of Applied Sciences
HW/SW Codesign Seminar Paper
Email: md.rafiul-islam@stud.hshl.de

***Abstract*—High-Level Synthesis converts an algorithmic description into register-transfer-level hardware. One central task in High-Level Synthesis is scheduling, where operations are assigned to clock cycles while data dependencies and limited hardware resources are respected. This paper studies Integer Linear Programming based scheduling for High-Level Synthesis. The scheduling problem is formulated using binary operation-to-cycle decision variables. The model includes operation uniqueness, dependency, resource, and latency constraints. A C++ implementation demonstrates the main feature of an ILP-style scheduler using the arithmetic expression y equals a multiplied by b plus c multiplied by d. Two resource configurations are evaluated: one multiplier with one adder, and two multipliers with one adder. The results show that increasing the number of multipliers reduces the latency from three clock cycles to two clock cycles. Finally, both schedules are realized as register-transfer-level datapaths in VHDL and verified in Questa. The paper shows how a mathematical scheduling decision influences the final RTL behavior, resource usage, and latency.**

***Index Terms*—High-Level Synthesis, Integer Linear Programming, scheduling, resource constraints, RTL, VHDL, C++.**

I. Introduction

Modern digital systems are increasingly complex. Designing such systems manually at the gate level or directly at the register-transfer level is time-consuming and error-prone. High-Level Synthesis, abbreviated as HLS, addresses this problem by allowing a designer to start from a behavioral description and automatically generate a hardware architecture. The behavioral description may be written in a high-level language such as C, C++, or SystemC. The HLS tool then transforms the behavior into datapath components, registers, multiplexers, and a controller [2], [3], [6].

One of the most important internal steps of HLS is scheduling. Scheduling decides when each operation of a behavioral description is executed. For example, multiplication, addition, comparison, and memory access operations must be assigned to clock cycles. This assignment is not arbitrary because operations may depend on results from earlier operations. Furthermore, the target architecture may contain only a limited number of functional units. Therefore, the scheduler must respect both data dependencies and resource constraints [1], [2].

The scheduling decision has a direct effect on the quality of the final hardware design. A schedule that uses more parallel operations can reduce latency, but it may require more hardware resources. A schedule that uses fewer hardware resources may reduce area, but it can increase the number of clock cycles. This trade-off between hardware cost and performance is one of the main reasons why scheduling is a central problem in HLS [1]–[3].

This seminar paper focuses on ILP-based scheduling for HLS. Integer Linear Programming, or ILP, is an exact mathematical optimization technique. It represents decisions by integer or binary variables and expresses the design rules as linear constraints. In the scheduling context, a binary variable can represent whether an operation is assigned to a specific clock cycle. The objective can be to minimize latency, minimize resources, or minimize cost under timing constraints [1], [2].

The main contribution of this paper is a compact demonstration of ILP-based HLS scheduling. First, the scheduling problem is described conceptually. Second, the mathematical ILP formulation is presented in a simplified form. Third, a C++ implementation demonstrates the main scheduling logic using a small arithmetic example. Finally, VHDL and Questa are used to show how the generated schedule can be realized as RTL hardware. This follows the seminar requirement that the topic should include mathematical basics, formal description, algorithmic explanation, runtime and overhead, C/C++ implementation, application example, and RTL realization [7].

II. High-Level Synthesis Overview

High-Level Synthesis is a design automation process that converts a behavioral algorithm into hardware. In a simplified view, the HLS flow starts with a high-level algorithm, extracts an internal operation representation, applies scheduling and resource decisions, and finally generates RTL hardware. The generated RTL usually contains functional units, registers, multiplexers, interconnections, and a controller [2], [6].

A functional unit performs an arithmetic or logical operation. Examples are adders, multipliers, comparators, and arithmetic logic units. Registers store intermediate results between clock cycles. Multiplexers route values to the correct functional units. The controller is often implemented as a finite state machine. It activates the correct operations and data transfers in each clock cycle.

HLS can be divided into several fundamental synthesis tasks. The first task is allocation. Allocation decides how many resources are available. For example, a design may allocate one multiplier and one adder, or it may allocate two multipliers and one adder. The second task is scheduling. Scheduling decides in which clock cycle each operation is executed. The

third task is binding. Binding assigns each scheduled operation to a concrete hardware resource instance [1], [2].

Although this paper focuses on scheduling, the scheduling result is strongly influenced by allocation. If the allocation contains two multipliers, then two independent multiplication operations can execute in the same cycle. If the allocation contains only one multiplier, those operations must execute in different cycles. Therefore, scheduling and allocation together determine the latency and resource cost of the resulting architecture.

A key idea in HLS is that the same behavioral expression can lead to different hardware architectures. A fast design may use more functional units and finish in fewer clock cycles. A smaller design may reuse the same functional unit across multiple cycles. The scheduler is responsible for making this time-resource trade-off explicit.

III. Scheduling Problem in HLS

The scheduling problem can be represented using an operation graph. The graph is directed. Each node represents an operation, and each directed edge represents a data dependency. If there is an edge from one operation to another, the second operation cannot start until the first operation has produced its result [1], [2].

The application example used in this paper is the arithmetic expression y equals a multiplied by b plus c multiplied by d. This expression contains three operations. Operation O1 computes m1 equals a multiplied by b. Operation O2 computes m2 equals c multiplied by d. Operation O3 computes y equals m1 plus m2. The addition operation O3 depends on both multiplication results. Therefore, O3 cannot start before O1 and O2 have finished.

The dependency relation can be written conceptually as O1 before O3 and O2 before O3. There is no dependency between O1 and O2. This means the two multiplication operations are independent. If enough hardware is available, they can execute in parallel. If only one multiplier is available, they must execute sequentially.

This example is small, but it captures the core scheduling problem. The scheduler must decide whether operations can run together, whether their dependencies are satisfied, and whether the available hardware resources are sufficient. The final result is a schedule that maps operations to clock cycles.

For the example expression, two resource configurations are considered. In the first configuration, there is one multiplier and one adder. Since there is only one multiplier, the two multiplication operations cannot be scheduled in the same clock cycle. In the second configuration, there are two multipliers and one adder. Since the two multiplications are independent, both can execute in the first clock cycle. This difference produces different latencies.

IV. Mathematical Basics

The operation graph can be written as G equals V and E, where V is the set of operation nodes and E is the set of dependency edges. Each operation has an operation type. In this paper, the relevant operation types are multiplication and addition. Each operation is assigned to a clock cycle or control step.

Let R represent a resource type. For example, all multiplication operations require a multiplier resource, and all addition operations require an adder resource. Let A represent the number of available hardware units of a resource type. If only one multiplier is available, only one multiplication can be executed in a single clock cycle. If two multipliers are available, two multiplication operations may execute in the same cycle.

The scheduling problem has four main requirements. First, every operation must be scheduled exactly once. Second, all data dependencies must be respected. Third, the number of operations assigned to one cycle must not exceed the available resources. Fourth, an optimization objective should be satisfied. In this paper, the objective is latency minimization.

Latency is the number of clock cycles required to finish the scheduled computation. A lower latency means the output is produced earlier. In HLS, latency is an important performance metric, especially when a design is used as a hardware accelerator or inside a real-time system.

The scheduling problem is therefore a constrained optimization problem. The designer wants the best schedule, but the schedule must obey the dependency and resource rules. ILP is suitable for this kind of problem because it can express binary decisions and linear constraints in one mathematical model [1], [2].

V. Formal ILP Model of Scheduling

ILP-based scheduling introduces binary decision variables. A variable x of i and t is equal to one if operation i starts at time step t. Otherwise, the variable is zero. This is the central modeling idea. Instead of directly guessing the start time of an operation, the model uses a set of yes-or-no variables for all possible operation-to-cycle assignments [1], [2].

The first constraint ensures that every operation is scheduled exactly once. This means that for each operation, the sum of all possible assignment variables must be exactly one. This constraint prevents two invalid cases. It prevents an operation from being unscheduled, and it also prevents the same operation from being scheduled more than once.

The start time of an operation can be calculated from the binary variables. Since only one assignment variable for an operation can be one, the summation selects the actual clock cycle assigned to that operation.

For every dependency edge, the successor operation must start after the predecessor operation finishes. If an operation produces a value that is used by another operation, the consumer operation must not be scheduled before the producer operation. This is the mathematical representation of data dependency.

The resource constraint ensures that the number of operations of a given type in the same cycle does not exceed the number of available functional units. For example, if only one multiplier is available, the scheduler must not assign two

multiplication operations to the same cycle. If two multipliers are available, two multiplications may be allowed in the same cycle.

The latency is the maximum completion time of all operations. The objective function used in this paper is to minimize this latency. Therefore, the ILP model tries to find the shortest valid schedule under the available resource constraints.

This model captures the core idea of ILP-based scheduling. Binary variables represent operation-to-cycle decisions. Dependency constraints guarantee correctness. Resource constraints model hardware limitations. The objective function selects the schedule with the smallest latency.

VI. Algorithm Steps

The implemented algorithm follows the same logical structure as the ILP model. First, the operation graph is defined from the input expression. The program stores operation identifiers, operation types, and textual descriptions. Second, dependency edges are stored. Third, the available resources are defined. In the experiments, the number of adders is fixed to one, while the number of multipliers is changed from one to two.

After the input has been created, the program enumerates possible operation-to-cycle assignments. For each candidate schedule, the implementation checks whether every operation has been scheduled. It then checks the dependency constraints. If an addition is scheduled before its required multiplication results are available, the candidate schedule is rejected. The program also checks resource constraints in every cycle. If two multiplication operations are assigned to the same cycle while only one multiplier is available, the candidate schedule is rejected.

After a candidate passes all constraint checks, the program computes the latency. The best feasible schedule is the feasible schedule with the smallest latency. This is equivalent to the objective of minimizing the final completion time.

A full industrial ILP-based HLS tool would normally generate the mathematical model and pass it to an ILP solver. Examples of professional solvers include CPLEX or Gurobi. In this seminar implementation, an exhaustive ILP-style search is used instead. This choice makes the implementation simple and transparent. The goal is not to build an industrial HLS tool, but to demonstrate the main scheduling concept in C++.

The algorithm is exact for the small search space used in the example. However, exhaustive enumeration is not scalable for large graphs. This limitation is discussed in the runtime and overhead section.

VII. Runtime and Overhead

ILP-based scheduling is an exact optimization approach, but exactness has a computational cost. The number of binary variables depends on the number of operations and the number of possible time steps. If a graph has many operations and each operation can be placed in many time steps, the number of variables grows quickly.

The number of constraints also grows with the problem size. Operation uniqueness constraints grow with the number of operations. Dependency constraints grow with the number of edges in the operation graph. Resource constraints grow with the number of resource types and time steps. Therefore, large designs can produce large ILP models.

For small examples, ILP is practical and useful. It gives an optimal answer for the defined objective. For large examples, the runtime can become high. This is why heuristic algorithms such as list scheduling or force-directed scheduling are also widely studied in HLS [2], [4]. These heuristics are usually faster, but they do not always guarantee an optimal schedule.

The runtime-overhead trade-off can be summarized as follows. ILP gives strong mathematical correctness and optimality for the chosen formulation. However, its solving time can become large. Heuristic scheduling is faster and often suitable for large designs, but it may miss the optimal solution. Therefore, ILP-based scheduling is especially useful for small designs, critical kernels, academic analysis, or architecture exploration.

VIII. C/C++ Implementation

The main implementation was developed in C++. The program represents the operation graph, dependencies, and resource constraints. It then searches for feasible schedules and selects the one with minimum latency. This follows the seminar requirement that the main feature should be implemented in C or C++ [7].

```
PS D:\Desktop\Real time & Hardware\achim rettberg\ILP_HLS_Scheduling> .\ilp_hls_scheduler.exe
==================================================
ILP-BASED SCHEDULING FOR HIGH-LEVEL SYNTHESIS
==================================================

Application example:
    y = (a * b) + (c * d)

Operations:
    O1 = m1 = a * b   Type: MUL
    O2 = m2 = c * d   Type: MUL
    O3 = y  = m1 + m2   Type: ADD

Dependencies:
    O1 -> O3
    O2 -> O3

Resource constraints:
    Available multipliers = 1
    Available adders      = 1

ILP binary decision variable:
    x[i][t] = 1 if operation i starts in clock cycle t
    x[i][t] = 0 otherwise

ILP constraints checked by the program:
    1. Each operation is scheduled exactly once.
    2. Data dependencies must be respected.
    3. Resource limits must not be exceeded in any cycle.
    4. Objective: minimize total latency.

Search horizon: 1 to 5 cycles
--------------------------------------------------

Number of feasible schedules checked = 2
```

Fig. 1. C++ implementation overview showing the application example, operations, dependencies, resource constraints, binary decision variable, and ILP-style constraints.

The C++ program uses three operations. Operation O1 is a multiplication and computes m1 equals a multiplied by b. Operation O2 is also a multiplication and computes m2 equals c multiplied by d. Operation O3 is an addition and computes y equals m1 plus m2. The dependency structure is also stored explicitly. The program stores the edges from O1 to O3 and from O2 to O3.

The implementation checks four main conditions. First, each operation must be scheduled exactly once. Second, all data dependencies must be respected. Third, the number of

operations assigned to a cycle must not exceed the available hardware resources. Fourth, the program selects the schedule with minimum latency.

Although the program does not call an external solver, it still demonstrates the ILP scheduling concept. The variables correspond to binary operation-to-cycle decisions, and the constraint checks correspond to the formal ILP constraints. This makes the implementation useful for explaining the theory in a practical and visible way.

IX. Application Example and Results

Two resource configurations were evaluated. The first case uses one multiplier and one adder. Because only one multiplier is available, the two multiplication operations cannot run in the same cycle. Therefore, they are scheduled sequentially.

```
Dependencies:
    O1 -> O3
    O2 -> O3

Resource constraints:
    Available multipliers = 1
    Available adders      = 1

ILP binary decision variable:
    x[i][t] = 1 if operation i starts in clock cycle t
    x[i][t] = 0 otherwise

ILP constraints checked by the program:
    1. Each operation is scheduled exactly once.
    2. Data dependencies must be respected.
    3. Resource limits must not be exceeded in any cycle.
    4. Objective: minimize total latency.

Search horizon: 1 to 5 cycles
------------------------------------------------------

Number of feasible schedules checked = 2

Best feasible schedule found:

Cycle 1: O1 [MUL] m1 = a * b
Cycle 2: O2 [MUL] m2 = c * d
Cycle 3: O3 [ADD] y  = m1 + m2

Minimum latency = 3 clock cycles

Explanation:
    With the given resource limits, the scheduler assigns
    each operation to a control step while respecting data
    dependencies and hardware resource constraints.
PS D:\Desktop\Real time & Hardware\achim rettberg\ILP_HLS_Scheduling>
```

Fig. 2. C++ result for one multiplier and one adder. The multiplication operations are serialized and the minimum latency is three clock cycles.

For the first case, the resulting schedule is cycle 1 for O1, cycle 2 for O2, and cycle 3 for O3. The latency is three clock cycles. This result is expected because one multiplier can perform only one multiplication operation per cycle.

The second case uses two multipliers and one adder. Since O1 and O2 are independent, they can execute in parallel in the first cycle. The addition then executes in the second cycle.

```
ILP-BASED SCHEDULING FOR HIGH-LEVEL SYNTHESIS
================================================

Application example:
    y = (a * b) + (c * d)

Operations:
    O1 = m1 = a * b   Type: MUL
    O2 = m2 = c * d   Type: MUL
    O3 = y  = m1 + m2   Type: ADD

Dependencies:
    O1 -> O3
    O2 -> O3

Resource constraints:
    Available multipliers = 2
    Available adders      = 1

ILP binary decision variable:
    x[i][t] = 1 if operation i starts in clock cycle t
    x[i][t] = 0 otherwise

ILP constraints checked by the program:
    1. Each operation is scheduled exactly once.
    2. Data dependencies must be respected.
    3. Resource limits must not be exceeded in any cycle.
    4. Objective: minimize total latency.

Search horizon: 1 to 5 cycles
------------------------------------------------

Number of feasible schedules checked = 1

Best feasible schedule found:

Cycle 1: O1 [MUL] m1 = a * b | O2 [MUL] m2 = c * d
Cycle 2: O3 [ADD] y  = m1 + m2

Minimum latency = 2 clock cycles
```

Fig. 3. C++ result for two multipliers and one adder. The independent multiplications execute in parallel and the minimum latency is reduced to two clock cycles.

For the second case, the resulting schedule is cycle 1 for both O1 and O2, followed by cycle 2 for O3. The latency is two clock cycles. This result demonstrates the effect of resource allocation on scheduling. By increasing the number of multipliers from one to two, the scheduler can exploit parallelism in the data-flow graph.

The comparison is simple but important. Case 1 uses fewer resources and needs three cycles. Case 2 uses more resources and needs two cycles. This is the classic HLS trade-off between area and performance. A designer may choose the first schedule for a smaller design or the second schedule for a faster design.

X. RTL Realization in VHDL

The VHDL implementation realizes the schedules generated by the C++ scheduler. It does not solve the ILP problem again. Instead, it maps the schedule to RTL states and datapath operations. This is important because HLS ultimately produces RTL components such as datapaths and controllers [2], [7].

The VHDL design contains a finite state machine and registers for intermediate values. The input values used for simulation are a equals 2, b equals 3, c equals 4, and d equals 5. The expected result is 26 because 2 multiplied by 3 gives 6, and 4 multiplied by 5 gives 20. The final addition gives 26.

A generic parameter named NUM-MULTIPLIERS is used to select the resource configuration. If this parameter is one, the VHDL design follows the three-cycle schedule. If this parameter is two, the design follows the two-cycle schedule.

For the one-multiplier case, the state sequence is IDLE, first multiplication, second multiplication, addition, and FIN-

ISHED. This sequence corresponds to the schedule generated by the C++ implementation for one multiplier. For the two-multiplier case, the state sequence is IDLE, parallel multiplication, addition, and FINISHED. This sequence corresponds to the optimized schedule generated for two multipliers.

A combined testbench instantiates both VHDL configurations in the same simulation. This allows both cases to be compared in one waveform. Both designs receive the same input values. Both designs compute the same result, but the two-multiplier design finishes earlier.

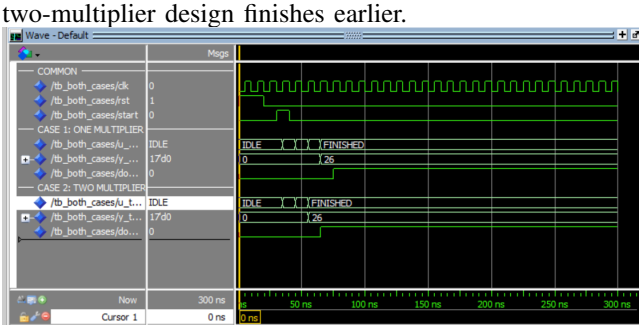

Fig. 4. Questa waveform comparing both RTL schedules. The one-multiplier design follows a three-cycle schedule, while the two-multiplier design finishes earlier using a two-cycle schedule. Both designs compute y equals 26.

The waveform shows that both RTL designs produce the correct result. The two-multiplier configuration finishes earlier because both multiplication operations are executed in parallel. This demonstrates the connection between the ILP schedule and the generated RTL behavior.

The VHDL result is not a separate scheduling algorithm. Instead, it is a hardware realization of the schedules found by the C++ scheduler. The C++ program acts as the planner, while the VHDL design acts as the hardware implementation. This distinction is important because ILP scheduling belongs to the HLS decision process, whereas VHDL describes the final RTL behavior.

XI. Advantages and Limitations

ILP-based scheduling has several advantages. It provides a formal mathematical optimization model. It can find an optimal schedule for the selected objective. It handles data dependencies explicitly. It handles resource constraints explicitly. It is also useful for comparing different hardware-resource configurations.

Another advantage is explainability. The decision variables and constraints can be inspected and related directly to the operation graph. This makes ILP useful in academic contexts and in design-space exploration. A designer can change the number of resources, change the latency objective, or add constraints and then observe how the schedule changes.

However, ILP-based scheduling also has limitations. The number of binary variables grows with the number of operations and time steps. Large designs may require high solving time. Industrial-scale designs usually require professional ILP solvers. More complex features such as loops, memory accesses, pipelining, branching, and multi-cycle operations increase model complexity.

For very large designs, heuristic algorithms may be more practical. List scheduling, force-directed scheduling, and other heuristic approaches can produce good schedules quickly, although they may not guarantee optimality [2], [4]. Therefore, ILP-based scheduling is valuable for exact optimization and architecture exploration, but it must be used carefully for large-scale HLS problems.

XII. Conclusion

This paper presented ILP-based scheduling for High-Level Synthesis. The scheduling problem was formulated using binary operation-to-cycle variables, dependency constraints, resource constraints, and a latency-minimization objective. A C++ implementation demonstrated the main concept using a small arithmetic expression.

The implementation compared two hardware configurations. With one multiplier and one adder, the two multiplication operations were serialized and the schedule required three cycles. With two multipliers and one adder, the independent multiplications were executed in parallel and the latency was reduced to two cycles.

The schedules were then realized in VHDL and simulated in Questa. The waveform confirmed that both RTL configurations compute the correct result, while the two-multiplier configuration finishes earlier. This shows how ILP-based scheduling decisions directly influence RTL hardware behavior, resource usage, and latency.

The main lesson is that ILP-based scheduling provides a clear mathematical way to reason about HLS scheduling. It connects an abstract operation graph to concrete hardware behavior. At the same time, its computational overhead means that it is most suitable for small or medium-sized problems, critical kernels, or cases where optimality is more important than solving time.

References

[1] J. Teich and C. Haubelt, *Digitale Hardware/Software-Systeme: Synthese und Optimierung*, 2nd ed. Berlin, Germany: Springer, 2007.

[2] G. De Micheli, *Synthesis and Optimization of Digital Circuits*. New York, NY, USA: McGraw-Hill, 1994.

[3] M. C. McFarland, A. C. Parker, and R. Camposano, "The high-level synthesis of digital systems," *Proceedings of the IEEE*, vol. 78, no. 2, pp. 301–318, Feb. 1990.

[4] P. G. Paulin and J. P. Knight, "Force-directed scheduling for the behavioral synthesis of ASICs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 8, no. 6, pp. 661–679, Jun. 1989.

[5] L. J. Hafer and A. C. Parker, "A formal method for the specification, analysis, and design of register-transfer level digital logic," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 2, no. 1, pp. 4–18, Jan. 1983.

[6] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, "High-level synthesis for FPGAs: From prototyping to deployment," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 30, no. 4, pp. 473–491, Apr. 2011.

[7] A. Rettberg, "HW/SW Codesign Seminar Organization and Elaboration," Hamm-Lippstadt University of Applied Sciences, lecture slides, 2026.